

Arch1

Cohort 3 Group 8

Henry Gillett, Rishi Thiahulan, Isaac Mileham, Ida Kenyon,
Lenny Schwarz, Viktor Barabash, Andrii Kuznietsov

Architecture Design

This document describes the software architecture of our team's 2D adventure game called "Escape from Uni". The architecture defines the main components of the system, their responsibilities, and how they interact to deliver gameplay features such as player movement, interaction with objects, event triggering, and win/loss conditions etc. This design is following object-oriented design principles and uses **UML** (Unified Modelling Language) to describe the relationship between general components as well as low level view of each class in the game.

Main Goals:

- **Modularity** - Each class or component should have a single, well-defined purpose.
- **Maintainability** - The game can be simply understood and its function via component diagrams
- **Scalability** - Modularity allows us to easily modify the game elements such as increasing difficulties, add more complex AI characters or events(*Ref. UR-14; FR-13*).
- **Interactivity** - The game should motivate the player to interact with the environment through events and interactions. (*Ref. FR-05;FR-10*)

Supported Goals:

- **Portability** - Architecture should allow it to be played on different platforms (Windows, Linux, MacOS) (*Ref. UR-01*)
- **Accessibility** - Game settings can be easily adjusted for players with special needs(colour blind) (*Ref. NFR-06; FR-12*)
- **Robustness** - The game should easily handle unexpected input and do not affect the game process (*Ref. NFR-08*)

Evidence of the early design process, including UML draft and event list example is available on our team's website [here](#) . (*Ref. FR-14*). We also came up with the events list, which we passed to the implementation team, where they decided which one of them they will implement in the game.

We developed two structural diagrams which are class and component diagrams to show scale of game architecture, as well as one behavioral diagram called activity diagram which illustrates the flow of control in a system. Diagrams are here to familiarize you with the main game concepts, how the game process flows, as well as look under the hood by looking at the main super- and sub- classes in the project.

Tool used: LUCID.APP

UML Type: Class Diagram (Figure 1)

Purpose: to show low-level modular architecture of the game with describing internal structures.

Description: The diagram below represents structural architecture of the game with corresponding relationships between elements.

Main Elements:

- **Game Manager** - Controls the overall game loop, win/loss conditions, interactions and increases player's score
- **Game State** - manages play, pause, victory conditions
- **PlayerCharacter** - main class represents player avatar including its position, movements, tools and its score
- **Map and Location** define the game world and the location within it
- **Object, Tool, Static/Interactive Objects** are in-game items and obstacles
- **Events** (Simple, Fun, SpeedUp) handles dynamic in-game actions which can affect the timer, score or just entertain the player
- **Arrows** shows the relationships between different classes of objects

–Colours: **Orange** - Superclass; **Grey** - Subclass

–Arrows represents the interaction between classes

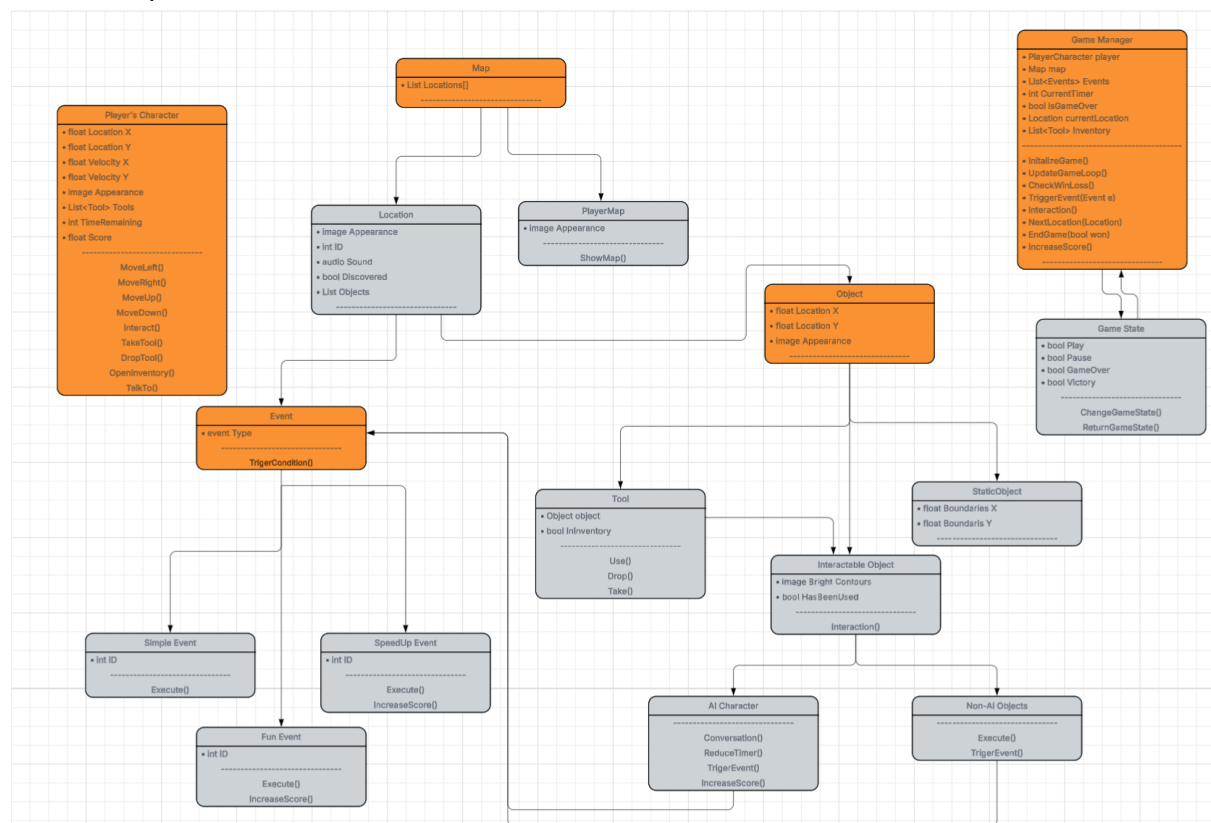


Figure 1

Tool used: LUCID.APP

UML Type: Component Diagram (Figure 2)

Purpose: to show high level modular architecture of the game, identifying major components and their interconnections

Description: The component diagram represents main subsystems of the game and how they interact with each other. Each “Manager” represents a modular unit responsible for specific game functionality.

Main Components:

- **Game Engine** - Controls game flow (time, win/lose, transitions)
- **Location Manager** - Loads and unloads rooms, handles transitions
- **Player Controller** - Handle input and player’s movement
- **Pause/Play Manager** - Stops the Game Engine when player needs a pause
- **Puzzle Manager** - Handle puzzles, progress, triggers
- **Inventory Manager** - Stores and uses collected items
- **Hint Manager** - Gives hints to player if he needs it or after some time
- **Event Manager** - Handle timings and places for events
- **AI Character** - handle spawns, interaction AI with player
- **Dialogue Manager** - displays narrative text, dialogue, receives responds
- **Item Manager** - Loads, spawns and manages collectible or interactive items
- **Audio Manager** - handles the sound for the game
- **UI Manager** - Controls UI screens(menus, inventory, map, hints)
- **Settings** - sound volume, controller configuration, UI settings
- **Camera Controller** - Manages camera focus

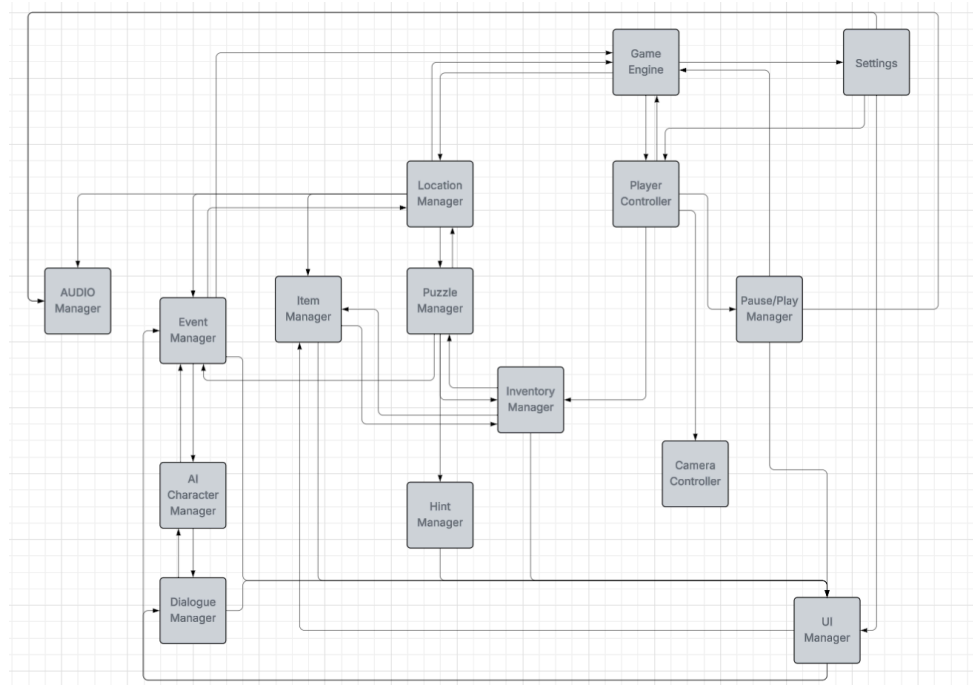


Figure 2

Tool used: LUCID.APP

UML Type: Activity Diagrams (Figure 3, 4)

Purpose: to model the sequence of player actions and system decisions during gameplay loop.

Description: The diagram (Figure 4) represents how the player should play each location and move towards leaving the location. The player can solve the puzzle without interacting with other elements such as AI characters or finding tools which will benefit the player in the future. At the same time, Figure 3 shows how the game engine will act every loop it goes through.

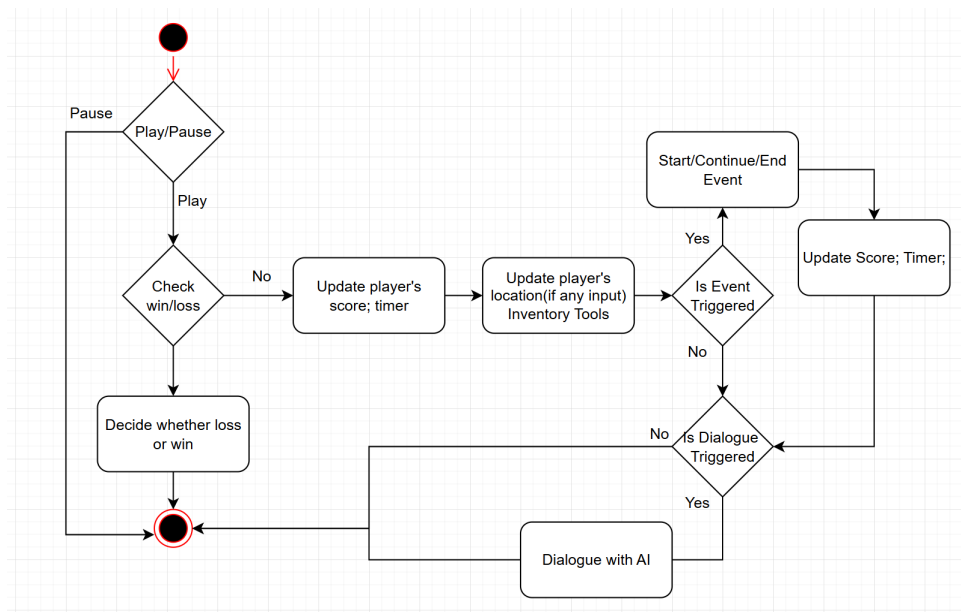


Figure 3

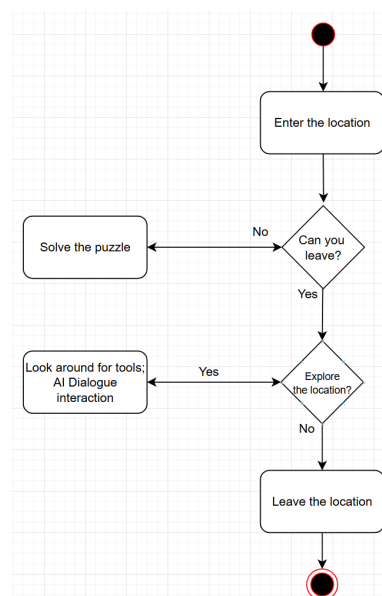


Figure 4

Regarding the **tutorial** part, the game will provide the manual tutorial, which includes photos of the keyboard and what keys are binded to with precise description, as well as the photos of how to generally behave in every location, such as where to move, what objects are interactable, etc.

Tutorial

When you launch the game, you'll first see a screen with instructions on how to play.

- W — move forward
- S — move backward
- A — move left
- D — move right
- E — interact
- E / Enter — advance dialogue with characters
- Y / N — choose in scenario with options

This document presents the software architecture of **Escape from Uni**. It describes the system's main components, their responsibilities, and their interactions that enable the key gameplay mechanics: movement, object interaction, event flow, and win/loss logic. The design follows object-oriented principles and employs a modular structure, which helps us achieve the goals of modularity, maintainability, scalability, and interactivity. The UML class and component diagrams (Figs. 1–2) and the gameplay activity diagrams (Figs. 3–4) make these decisions explicit and traceable—from low-level classes to high-level subsystems and gameplay loops.